

Day-2: File Ownership and Access Control

Topics: chown, chgrp, ownership + permissions

Part A - Ownership Investigation (10 Minutes)

- Create a directory named ownership_lab.
mkdir ownership_lab
- Inside it, create:
incident_report.txt
network.log
credentials.txt
cd ownership_lab
touch incident_report.txt network.log credentials.txt

- Display the detailed information for these files.
ls -l

- For each file, identify:

Owner
Group
Owner permissions
Group permissions
Others permissions

File	Owner	Group	Owner Permissions	Group Permissions	Others Permissions
incident_report.txt	shrutha	shrutha	Read, Write (rw-)	Read (r--)	Read (r--)
network.log	shrutha	shrutha	Read, Write (rw-)	Read (r--)	Read (r--)
credentials.txt	shrutha	shrutha	Read, Write (rw-)	Read (r--)	Read (r--)

- Determine the UID and GID of the current user.
UID: id -u
GID: id -g
- Compare the UID/GID information with the ownership displayed for the newly created files.
The UID and GID of the current user match the owner and group of the newly created files. This is because files are normally created with the current user as the owner and the user’s primary group as the group owner.
- Explain the difference between file ownership and file permissions.
File ownership specifies which user and group are associated with a file. File permissions specify the actions that the owner, group members, and other users can perform on the file, such as reading, writing, or executing it. Ownership identifies who owns the file, while permissions control what users can do with the file.

Part B - chown: Changing Ownership (15 Minutes)

- Change the owner of incident_report.txt to alice.
sudo chown alice incident_report.txt
- Change the owner of network.log to bob.
sudo chown bob network.log
- Change the owner of credentials.txt to charlie.
sudo chown charlie credentials.txt
- Verify each change.
ls -l <filename>
- Try changing the owner of a file to another user without using sudo. Record the result.
chown alice network.log
The ownership was not changed because an ordinary user does not have permission to transfer file ownership to another user.
- Explain why an ordinary user is generally not permitted to transfer file ownership to another user.
An ordinary user usually cannot change the ownership of a file because only an administrator has the permission to transfer files to other users. This helps prevent security issues and keeps file ownership under proper control. Therefore, the chown command usually requires sudo privileges.
- Using a single command, configure:
incident_report.txt
Owner: alice
Group: security
chown alice:security incident_report.txt
- Verify the resulting ownership.
ls -l incident_report.txt

Part C - chgrp: Changing Group Ownership (15 Minutes)

- Change the group ownership of network.log to developers.
sudo chgrp developers network.log
- Change the group ownership of credentials.txt to security.
sudo chgrp security credentials.txt
- Verify the changes.
ls -l network.log
ls -l credentials.txt
- Display the group memberships of alice, bob, and charlie.
groups alice
groups bob
groups charlie
- Log in as alice and create a file named alice_report.txt.
Login: su - alice
To create file: touch alice_report.txt
- Attempt to change the group of alice_report.txt to security.
chgrp security alice_report.txt
- Attempt to change its group to developers.
chgrp developers alice_report.txt
- Compare the results.
Alice was able to change the group of alice_report.txt to security, but Alice could not change it to developers. This is because Alice is a member of security but not developers.
- Explain how a user's group membership affects their ability to change the group ownership of a file they own.

A user who owns a file can generally change its group only to a group they belong to. Therefore, group membership determines which groups the user can assign to their own files.

Part D - Ownership + Permission Investigation (20 Minutes)

- Configure:
`-rw-r-----` alice security incident_report.txt
Configure the ownership: **sudo chown alice:security incident_report.txt**
Configure the permissions: **sudo chmod 640 incident_report.txt**
- Identify what operations are available to:
alice: Can read and modify the file. (owner)
Charlie: Can only read the file. (security)
bob: Can only read the file. (security)
- A user who belongs to neither security nor the file owner
A user who is neither the owner nor a member of the security group gets the others permissions.
Since the file has --- for others, the user cannot read, modify, or access the file.
- charlie belongs to security. Test whether Charlie can:
Read incident_report.txt: can read
Modify incident_report.txt: cannot modify
Delete incident_report.txt: cannot delete
Record the result of each operation.
- Change the permissions to:
`-rw-rw----`
sudo chmod 660 incident_report.txt
- Repeat the test as charlie.
Read incident_report.txt: can read
Modify incident_report.txt: can modify
Delete incident_report.txt: cannot delete
- Explain why Charlie's access changed even though the owner and group of the file did not change.
Charlie's access changed because the group permission was changed from r-- to rw-. Charlie is a member of the security group, so he could read the file before, but after the permission change he could also modify it. The owner and group of the file remained the same.
- Remove bob from the security group.
sudo gpasswd -d bob security

- Without changing the file's ownership or permissions, test Bob's access again.
cannot read, modify and delete.

su -bob

- Explain why Bob's effective permissions changed.

Bob's effective permissions changed because he was removed from the security group. Although the file still has the same owner, group, and permissions, Bob is no longer a member of the group. Therefore, he receives the others permissions (---) instead of the group's rw-permissions.